

Technische Hochschule Deggendorf

Fakultät Angewandte Informatik

Studiengang Bachelor Cyber-Security

Kurs: Security Engineering – SS 2026

Projektdokumentation

Secure DevOps (DevSecOps) – Sicherheitstechnische Betrachtung automatisierter CI/CD-Pipelines

Vorgelegt von:

Christof Renner (22301943)
Manuel Friedl (12306626)

Betreuer:

Prof. Dr. Martin Schramm

Datum: 25. April 2026

Inhaltsverzeichnis

1	Einleitung	3
1.1	Motivation und Problemstellung	3
1.2	Zielsetzung	3
1.3	Abgrenzung	3
1.4	Aufbau der Arbeit	3
2	Grundlagen	4
2.1	DevSecOps	4
2.2	Software Supply Chain Security	4
2.3	Frameworks	4
2.4	Security Engineering Framework	4
3	System unter Betrachtung	5
3.1	Architekturüberblick	5
3.2	Schützenswerte Assets	5
3.3	Trust Boundaries	6
3.4	Angreifermodell	6
4	Bedrohungsanalyse (STRIDE)	6
4.1	STRIDE-Mapping pro Pipeline-Stufe	6
4.2	Threat Walkthroughs	7
4.3	Vertiefung: Poisoned Pipeline Execution (PPE)	9
5	Gegenmaßnahmen und Implementierung	11
5.1	Source-Stufe	11
5.2	Build-Stufe	11
5.3	Test- und Scan-Stufe	11
5.4	Package- und Sign-Stufe	12
5.5	SBOM-Lebenszyklus	12
5.6	Deploy-Stufe	14
5.7	Auszug aus der CI-Implementierung	14
5.8	Mapping Bedrohung → Maßnahme	15
5.9	Fallstudie: tj-actions/changed-files (März 2025)	15
6	Evaluation	16
6.1	SLSA-Einstufung	16
6.2	OpenSSF Scorecard	17
6.3	Coverage der STRIDE-Kategorien	18
6.4	Validierung gegen OWASP Top 10 CI/CD	18
6.5	Restrisiken	19
6.6	Zielkonflikt Geschwindigkeit vs. Assurance	19
6.7	Faktor Mensch und Anreize	19
7	Fazit und Ausblick	21

1 Einleitung

1.1 Motivation und Problemstellung

Moderne Softwareentwicklung basiert auf hochgradig automatisierten Build-, Test- und Deployment-Prozessen. CI/CD-Pipelines sind tief in cloud-native Infrastrukturen integriert und verfügen über weitreichende Zugriffsrechte auf sensible Ressourcen wie Quellcode-Repositories, Secrets, Container-Registries und Produktionsumgebungen. Angriffe auf Softwarelieferketten – etwa SolarWinds (2020), Codecov (2021), Log4Shell (2021) und der 3CX-Vorfall (2023) – belegen, dass der Entwicklungsprozess selbst zu einem primären Angriffsziel geworden ist.

Aus Sicht des Security Engineering ist eine Pipeline weniger ein Tooling-Problem als ein *Systemproblem*: Sie verbindet menschliche Identitäten, externe Abhängigkeiten, Build-Hosts, Registries und Deployment-Targets über mehrere Vertrauensgrenzen hinweg. Eine isolierte Betrachtung einzelner Werkzeuge greift daher zu kurz.

1.2 Zielsetzung

Ziel der Arbeit ist eine systematische sicherheitstechnische Betrachtung einer repräsentativen CI/CD-Pipeline. Konkret werden:

- schützenswerte Assets und Trust Boundaries identifiziert,
- Bedrohungen je Pipeline-Stufe nach STRIDE strukturiert analysiert,
- geeignete Sicherheitsmechanismen abgeleitet und in einer prototypischen Pipeline (GitHub Actions) umgesetzt,
- die Wirksamkeit dieser Maßnahmen anhand von SLSA-Levels und OpenSSF-Scorecard evaluiert,
- verbleibende Restrisiken transparent gemacht.

1.3 Abgrenzung

Nicht Gegenstand der Arbeit sind: vollständige Härtung des Cloud-Tenants, Penetrationstests der Pipeline-Plattform selbst sowie organisatorische Maßnahmen jenseits der Branch-Protection-Konfiguration. Die Pipeline wird auf GitHub Actions umgesetzt; die Übertragbarkeit auf GitLab CI wird nicht im Detail untersucht und in Kapitel 7 lediglich als weiterführende Forschungsrichtung benannt.

1.4 Aufbau der Arbeit

Kapitel 2 klärt die Begriffe DevSecOps, Supply-Chain-Security und die zugrundeliegenden Frameworks (STRIDE, SLSA, SSDF). Kapitel 3 beschreibt das modellierte System und seine Trust Boundaries. Kapitel 4 enthält das STRIDE-Threat-Model. Kapitel 5 ordnet jeder Bedrohung konkrete Gegenmaßnahmen zu und dokumentiert die Implementierung

im Prototyp. Kapitel 6 bewertet die Wirksamkeit, Kapitel 7 fasst zusammen und diskutiert Restrisiken.

2 Grundlagen

2.1 DevSecOps

DevSecOps bezeichnet die Integration sicherheitstechnischer Aktivitäten in den DevOps-Lebenszyklus mit dem Ziel, Sicherheit als kontinuierlichen, automatisierten Bestandteil von Build und Deployment zu etablieren („shift left“ und „shift right“). Sicherheitsmaßnahmen werden nicht nachgelagert aufgesetzt, sondern als Teil derselben Pipeline ausgeführt, die auch das Artefakt produziert.

2.2 Software Supply Chain Security

Eine Software Supply Chain umfasst alle Personen, Prozesse, Werkzeuge und Abhängigkeiten, die zur Erstellung und Auslieferung eines Artefakts beitragen. Angriffsklassen nach [1] sind insbesondere:

- **Source**-Angriffe (z. B. kompromittierte Maintainer-Accounts, Typosquatting),
- **Build**-Angriffe (z. B. kompromittierte Runner, manipulierte Compiler/Plugins),
- **Dependency**-Angriffe (z. B. Übernahme verwaister Pakete),
- **Distribution**-Angriffe (z. B. Image-Tag-Replacement in Registries).

2.3 Frameworks

STRIDE

Microsoft-Methodik zur Bedrohungsmodellierung mit den Kategorien *Spoofing*, *Tampering*, *Repudiation*, *Information Disclosure*, *Denial of Service*, *Elevation of Privilege*.

SLSA v1.0

Supply-chain Levels for Software Artifacts definiert Anforderungen an Build-Integrität in vier Stufen (Build L1–L3, Source L1–L3).

NIST SSDF (SP 800-218)

Secure Software Development Framework – Praktiken über den gesamten SDLC.

OpenSSF Scorecard

Automatisierte Bewertung von Repository-Posture anhand objektiver Checks.

2.4 Security Engineering Framework

Die Arbeit folgt dem in der Lehrveranstaltung eingeführten Vier-Quadranten-Modell des Security Engineering nach Anderson: *Policy*, *Mechanism*, *Assurance* und *Incentives*. Ta-

belle 1 ordnet jedem Quadranten konkrete Bestandteile der untersuchten Pipeline zu; auf den Quadranten *Incentives* wird in Abschnitt 6.7 vertiefend eingegangen.

Quadrant	Umsetzung in der Pipeline
Policy	Branch-Protection-Regeln, CODEOWNERS, Required Status Checks, Coverage-Schwelle 80 %, HIGH/CRITICAL als Build-Breaker, Manual-Approval-Gate für Production.
Mechanism	Bandit, CodeQL, pip-audit, Trivy, Syft (SBOM), Cosign Keyless Signing, SLSA-Provenance, OIDC-Federation, Action-Pinning auf Commit-SHA.
Assurance	SLSA-Level-Bewertung, OpenSSF Scorecard (wöchentlich), SARIF-Upload in Code-Scanning, Cosign-Verify im CD-Workflow, Sigstore Rekord Transparency Log.
Incentives	Vier-Augen-Prinzip via CODEOWNERS, automatisierte Dependabot-PRs reduzieren Reibung für sichere Updates, parallele Jobs + <code>cancel-in-progress</code> mildern den Geschwindigkeits-Konflikt (siehe Abschnitt 6.7).

Tabelle 1: Mapping der Pipeline-Bestandteile auf das Security-Engineering-Framework.

3 System unter Betrachtung

3.1 Architekturüberblick

Untersucht wird eine prototypische, container-basierte Pipeline für eine Python-Webanwendung. Die wesentlichen Stufen sind in Abbildung 1 dargestellt.

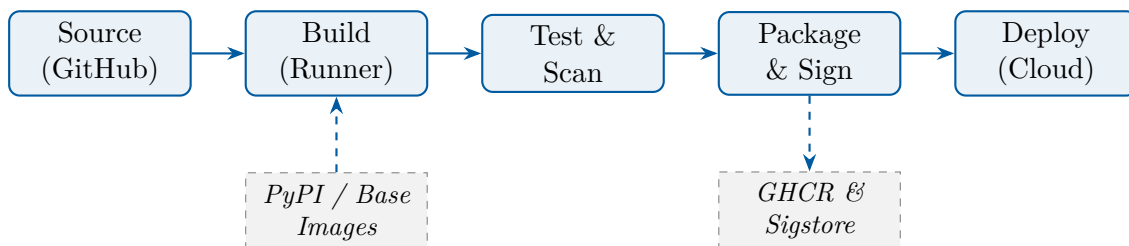


Abbildung 1: Pipeline-Stufen und externe Vertrauensanker.

3.2 Schützenswerte Assets

Asset	Schutzziel (CIA + AAA)
Quellcode (main)	Integrität, Authentizität
Build-Artefakt (Image)	Integrität, Authentizität, Verfügbarkeit
Secrets (Cloud-Cred., Tokens)	Vertraulichkeit, Integrität
SBOM & Provenance	Integrität, Nicht-Abstreitbarkeit

Asset	Schutzziel (CIA + AAA)
Pipeline-Konfiguration	Integrität, Autorisierung
Build-Runner-Identität (OIDC)	Authentizität, Vertraulichkeit

3.3 Trust Boundaries

1. Entwickler-Workstation ↔ GitHub
2. GitHub-Plattform ↔ Build-Runner
3. Build-Runner ↔ Paket-/Image-Registries (PyPI, Docker Hub, GHCR)
4. Build-Runner ↔ Sigstore (Fulcio/Rekor)
5. Pipeline ↔ Deployment-Target (Cloud-API)

3.4 Angreifermodell

Drei relevante Profile werden betrachtet:

A1 – Externer Angreifer

kein Repo-Zugriff; nutzt Phishing, Typosquatting, Schwachstellen in Abhängigkeiten.

A2 – Kompromittierter Mitwirkender

gültige, aber begrenzte Schreibrechte (z. B. Fork-PR, einzelner Maintainer).

A3 – Insider/Plattform

Admin-Rechte am Repo bzw. kompromittierter Self-hosted-Runner.

4 Bedrohungsanalyse (STRIDE)

4.1 STRIDE-Mapping pro Pipeline-Stufe

Die folgende Tabelle dokumentiert pro Pipeline-Stufe die wesentlichen Bedrohungen, klassifiziert nach STRIDE. Die Risiko-Einstufung folgt einer qualitativen Skala (niedrig/mittel/hoch) auf Basis von Eintrittswahrscheinlichkeit und Schadensausmaß im Kontext einer Studienarbeitstypischen Demo-Pipeline.

Stufe	STRIDE	Bedrohung	Angr.	Risiko
Source	S	Push mit gestohlenen Maintainer-Credentials	A1/A2	hoch
Source	T	Manipulation von Pipeline-Definitionen via PR	A2	hoch
Source	R	Unsignierte Commits, fehlende Audit-Trails	A2	mittel

Stufe	STRIDE	Bedrohung	Angr.	Risiko
Source	I	Secrets im Klartext im Repo (Logs, Con-figs)	A1	hoch
Build	T	Bösartige transitive Dependency (Confused-Deps, Typosquat)	A1	hoch
Build	T	Manipulation des Base-Images (Tag-Replacement)	A1	hoch
Build	E	Workflow mit zu weiten permissions (RW everywhere)	A2	hoch
Build	E	pull_request_target mit secrets ausgesetzt	A1	hoch
Build	I	Cache-Poisoning zwischen PRs	A2	mittel
Test	D	Lange laufende Scans blockieren Deployments	A1	niedrig
Test	T	Test-Bypass via geänderte Workflow-Bedingungen	A2	mittel
Pkg	T	Ungesignedes Image, Tag-Hijack in Registry	A1	hoch
Pkg	R	Fehlende Provenance / SBOM	–	mittel
Deploy	S	Long-lived Cloud-Credentials in Repo-Secrets	A2	hoch
Deploy	E	Fehlende Approval-Gates für Production	A2	hoch
Deploy	I	Verbose Logs leaken Tokens / PII	A1	mittel

Tabelle 3: STRIDE-Mapping der untersuchten Pipeline.

4.2 Threat Walkthroughs

Die kondensierte Tabellenform aus Tabelle 3 verdichtet Bedrohungen, verschleiert aber ihre tatsächliche *Verkettung*. Reale Angriffe auf CI/CD-Systeme bestehen selten aus einem einzelnen Schritt; charakteristisch ist die schrittweise Eskalation entlang der Trust Boundaries. Im Folgenden werden drei repräsentative Bedrohungen aus Tabelle 3 als ausformulierte Angriffsketten dargestellt – inklusive Voraussetzungen, Eskalationsschritten, Erkennungs- und Mitigationpunkten.

Walkthrough 1 – Bösartige transitive Dependency (Build-T, A1). *Voraussetzung:* Eine populäre, indirekt eingezogene Abhängigkeit wird durch einen kompromittierten Maintainer-Account aktualisiert; die neue Version enthält eine Post-Install-Routine, die Umgebungsvariablen und `~/.aws/credentials` an einen externen HTTP-Endpunkt sendet.

Schritt 1 – Aufnahme. Im Zielprojekt erstellt Dependabot plangemäß einen wöchentlichen Update-PR. Reviewer prüfen den Diff der `requirements.txt`, sehen jedoch nur einen scheinbar harmlosen Versionssprung der direkten Abhängigkeit; die *transitive* Aktualisierung ist im Diff nicht sichtbar.

Schritt 2 – Build-Stufe. Der CI-Runner installiert die Abhängigkeit; die Post-Install-

Routine läuft mit den Umgebungs-Permissions des Workflows.

Schritt 3 – Exfiltration. Der Schadcode greift auf `$GITHUB_TOKEN` und auf etwaig gemountete Cloud-Credentials zu und sendet sie an den Angreifer-Endpunkt.

Erkennung. `pip-audit` prüft die finale Resolverliste gegen die OSV-Datenbank, fängt aber nur Versionen mit bereits öffentlich bekannter CVE. Eine *frische* Kompromittierung wird in den ersten Stunden nicht erkannt – hier ist die zweite Verteidigungslinie nötig: SBOM-Diff (Abschnitt 5.5) hebt das neue Paket sichtbar hervor, und Egress-Restriktionen auf dem Runner können den Exfiltrationsversuch blockieren.

Mitigation in der Pipeline. Drei Maßnahmen wirken zusammen: (1) `permissions: {}` als Default reduziert die Reichweite des `GITHUB_TOKEN`; (2) `persist-credentials: false` bei `actions/checkout` verhindert, dass das Token im `.git/config` liegen bleibt; (3) die Cloud-Federation per OIDC ohne langlebige Secrets im Repo entzieht dem Angreifer das wertvollste Ziel.

Restrisiko. Der Angriff bleibt möglich, solange transitive Updates nicht zwingend an einer kuratierten Allowlist hängen; eine weitergehende Härtung wäre der Einsatz eines Privat-Mirrors mit Repository-Manager (z. B. Sonatype Nexus) und manueller Freigabe neuer Versionen.

Walkthrough 2 – Manipulation der Pipeline-Definition via PR (Source-T, A2). *Voraussetzung:* Ein Kontributor mit „Triage“-Rolle (oder ein Forker mit angenommenem PR-Template) reicht einen unscheinbaren Pull-Request ein, der nicht den Anwendungscode, sondern eine Datei unter `.github/workflows/` ändert – etwa um einen zusätzlichen Schritt einzufügen, der `secrets.AWS_DEPLOY_KEY` in einen publizistischen Webhook schreibt.

Schritt 1 – Tarnung. Der PR ist optisch ein „Refactoring“: Der Workflow wird in mehrere Teildateien zerlegt, dabei werden zusätzliche `run:-`Schritte eingefügt. Reviewer ohne explizit gepflegte CODEOWNERS-Regeln neigen dazu, den Diff zu überfliegen.

Schritt 2 – Trigger. Der Angreifer wartet darauf, dass ein Maintainer den PR mergt. Sobald der Workflow auf `main` läuft, führt der eingefügte Schritt aus – mit den vollen `secrets.~`-Werten der Pipeline.

Erkennung. CODEOWNERS auf `/.github/` verschiebt den Review zwingend an die im Repo deklarierten Pipeline-Verantwortlichen; OpenSSF Scorecard markiert das Fehlen dieser Regel als Posture-Defizit. `actionlint` und `zizmor` können in der CI selbst auf typische PPE-Antipatterns prüfen.

Mitigation in der Pipeline. CODEOWNERS auf `/.github/`, Required-Status-Checks für jede Pipeline-Änderung, sowie die Regel, dass `secrets` nur in Jobs verfügbar sind, die einem Environment mit Approval zugeordnet sind.

Restrisiko. Ein Insider mit Admin-Rechten kann CODEOWNERS selbst ändern. Hier hilft nur Org-weites Audit-Logging mit Alarm bei Änderungen an Branch-Protection-Konfigurationen.

Walkthrough 3 – Tag-Hijack in der Registry (Pkg-T, A1). *Voraussetzung:* Der Angreifer hat sich Schreibzugriff auf den Registry-Namespace verschafft – z.B. über einen kompromittierten Service-Account-Token, der versehentlich in einem öffentlichen Issue gepostet wurde.

Schritt 1 – Repush. Der Angreifer baut lokal ein böses Image, vergibt denselben Tag wie das letzte Release (`ghcr.io/org/app:1.4.2`) und pusht es. Der Manifest-Digest ändert sich; der Tag zeigt jetzt auf das neue Manifest.

Schritt 2 – Pull bei Deployment. Ein Deployment-Skript, das nur den Tag referenziert, zieht beim nächsten Rollout das manipulierte Image.

Erkennung. Der CD-Workflow verifiziert vor dem Rollout per `cosign verify` sowohl die Signatur (gegen die OIDC-Identität des CI-Workflows) als auch die SLSA-Provenance-Attestation. Beide Checks schlagen für das manipulierte Image fehl, da der Angreifer keine gültige Signatur unter dem CI-Workflow-OIDC-Issuer erzeugen kann; das Rekorder-Transparency-Log macht solche Manipulationen darüber hinaus öffentlich auditierbar.

Mitigation in der Pipeline. Im CD-Workflow erfolgt die Verifikation *vor* jedem Deploy; die Deploy-Schritte referenzieren ausschließlich den Image-Digest (`@sha256:...`), nicht den Tag.

Restrisiko. Ein Angriff auf Sigstore selbst ist nicht unmittelbar mitigierbar – daher die Listung von Sigstore als Vertrauensanker in Abschnitt 3.

4.3 Vertiefung: Poisoned Pipeline Execution (PPE)

Die Bedrohungen *Build-T* (Manipulation Pipeline-Definition) und *Build-E* (`pull_request_target` mit Secrets) aus Tabelle 3 sind manifestationen einer übergeordneten Klasse, die OWASP als *Poisoned Pipeline Execution* [6] beschreibt. PPE bezeichnet jeden Angriff, der den „Konfigurations-als-Code“-Charakter moderner CI-Systeme ausnutzt: Wer den Workflow ändern darf, kontrolliert – transitiv – alle Aktionen, die in seinem Kontext ablaufen.

Direkte und indirekte PPE. Direkte PPE liegt vor, wenn der Angreifer die Pipeline-Definition selbst modifiziert (z.B. als Maintainer mit Push-Recht). Indirekte PPE nutzt Trigger aus, die *trotz* fehlender Schreibrechte Code im Kontext der Pipeline ausführen. Der prominenteste Vektor in GitHub Actions ist der Trigger `pull_request_target`.

Anatomie von `pull_request_target`. Im Gegensatz zum Standard-Trigger `pull_request` – der den Workflow im Kontext des PR-Branches ohne Zugriff auf `secrets` ausführt – läuft `pull_request_target` im Kontext des Ziel-Branches *mit* vollem Secret-Zugriff. Der Trigger wurde geschaffen, um Aktionen wie automatisches Labeling auch für Forks zu ermöglichen, bei denen der Forker keinen Schreibzugriff auf das Ziel-Repository besitzt. Genau diese Kombination (Code-Quelle = Fork, Secrets = Original-Repo) ist die Kernschwäche.

Ein typischer fehlerhafter Workflow checkt zusätzlich den PR-HEAD aus und führt dort Skripte aus:

```

1 on:
2   pull_request_target:      # laeuft im Ziel-Repo-Kontext mit Secrets
3     types: [opened, synchronize]
4 jobs:
5   ci:
6     steps:
7       - uses: actions/checkout@<sha>    # OK: HEAD = Ziel-Branch
8         with:
9           ref: ${github.event.pull_request.head.sha} # PROBLEM!
10      - run: npm ci && npm test          # PROBLEM: laedt + fuehrt Fork-
                                         Code aus

```

Listing 1: Anti-Pattern: `pull_request_target` mit unsicherem Checkout

Ein böartiger Forker fügt seinem PR-Branch einen `postinstall`-Hook in der `package.json` hinzu, der die Werte aller Umgebungsvariablen exfiltriert. Beim ersten `npm ci` im Workflow läuft dieser Hook – inklusive Zugriff auf `secrets.NPM_TOKEN`, `secrets.AWS_*` und das `GITHUB_TOKEN` mit Schreibrecht auf das Original-Repo.

Codecov 2021 als Realbeispiel. Im April 2021 wurde der Codecov-Bash-Uploader durch einen Fehler in einem Docker-Image-Build kompromittiert; in der Folge konnten Angreifer rund zwei Monate lang Umgebungsvariablen aus CI-Umgebungen exfiltrieren, die den Uploader integriert hatten. Der Vorfall war zwar kein klassischer PPE-Fall, illustriert aber die gleiche Kausalkette: ein in einem Pipeline-Schritt ausgeführtes Skript, das implizit Zugriff auf alle Secrets erhält, die im Kontext des Workflows verfügbar sind. Branchenweite Reaktion war eine Auseinandersetzung mit dem Trigger-Modell und der Secret-Sichtbarkeit.

Mitigation in der hier umgesetzten Pipeline. Die folgenden konstruktiven Entscheidungen schließen die PPE-Klasse weitgehend:

- **Kein `pull_request_target`.** Der CI-Workflow verwendet ausschließlich den sicheren `pull_request`-Trigger; Aktionen mit Secret-Bedarf (z. B. Image-Push) werden bei PRs per `if: github.event_name != 'pull_request'` ausgesperrt.
- **CODEOWNERS auf `/.github/`.** Jede Workflow-Änderung erfordert ein Review der designierten Pipeline-Verantwortlichen; das Vier-Augen-Prinzip wird damit auf die kritischste Datei-Klasse angewendet.
- **Default-deny permissions: `{}`.** Selbst wenn ein PPE-Angriff Code im Workflow-Kontext zur Ausführung bringt, ist das verfügbare `GITHUB_TOKEN` ohne Schreibrechte und kann weder Branches noch Releases manipulieren.
- **OIDC-Federation statt langlebiger Secrets.** Cloud-Zugriffe verwenden kurzlebige, an die OIDC-Identität des Workflows gebundene Tokens. Selbst eine erfolgreiche Exfiltration nutzt dem Angreifer wenig, da die Tokens auf Minutenbasis ablaufen und an die Workflow-Run-ID gebunden sind.
- **`persist-credentials: false`.** Der `GITHUB_TOKEN` wird nach `actions/checkout` nicht in `.git/config` verewigt; ein nachgelagertes Skript hat damit keinen Hebel mehr.

Verbleibende PPE-Risiken. Eine vollständige Immunität ist nicht erreichbar. Eine Kompromittierung der GitHub-Plattform selbst, eine Schwachstelle in einer als Dependency genutzten Action (vgl. Fallstudie `tj-actions` in Abschnitt 5.9) oder ein Insider mit Administrationsrechten am Repo bleiben außerhalb der Reichweite der hier diskutierten Maßnahmen. Diese Risiken werden in Abschnitt 6.5 explizit als Restrisiken geführt.

5 Gegenmaßnahmen und Implementierung

5.1 Source-Stufe

Branch Protection & CODEOWNERS

`main` ist geschützt; Merges erfordern ein Review durch Code-Owner (Vier-Augen-Prinzip), bestandene Pflicht-Checks, signierte Commits und lineare History.

Signierte Commits

Entwickler signieren mit SSH/GPG; nicht-signierte Commits werden serverseitig abgelehnt.

Secret-Scanning

`gitleaks` läuft als Pre-Commit-Hook und in der CI; GitHub Push-Protection ergänzt das.

Pinned Actions

Alle GitHub-Actions sind auf Commit-SHA gepinnt, um Tag-Retag-Angriffe (z. B. `tj-actions/changed-files` 03/2025) zu verhindern.

5.2 Build-Stufe

Least-Privilege-Permissions

Workflow-Default ist `permissions: {}`; jeder Job opt-in mit minimalen Scopes.

Ephemere Runner

Nutzung von GitHub-hosted Runnern (frische VM pro Job); Self-hosted bewusst nicht verwendet.

Reproduzierbares Image

Multi-stage `Dockerfile` mit gepinntem Base-Image, Non-Root-User (UID 10001), Read-only-Filesystem-tauglich.

Dependency-Pinning

`requirements.txt` mit exakten Versionen; Updates über Dependabot-PRs, die die volle Pipeline durchlaufen.

5.3 Test- und Scan-Stufe

- **SAST:** Bandit (Python) + GitHub CodeQL, Ergebnisse als SARIF in die GitHub Code-Scanning-API.

- **SCA:** `pip-audit` gegen die OSV-Datenbank, `-strict` bricht den Build bei bekannter CVE ab.
- **Container-Scan:** Trivy auf das gebaute Image, `HIGH,CRITICAL` mit `exit-code: 1`.
- **Coverage-Gate:** `pytest -cov-fail-under=80`.

5.4 Package- und Sign-Stufe

SBOM

Syft erzeugt CycloneDX-JSON aus dem Image; das SBOM wird als Attestation an das Image angeheftet.

Cosign Keyless Signing

Signatur via Sigstore Fulcio mit dem OIDC-Token des Workflows; keine langlebigen Schlüssel im Repo.

SLSA-Provenance

`actions/attest-build-provenance` erzeugt eine verifizierbare Provenance-Attestation (SLSA v1).

5.5 SBOM-Lebenszyklus

Eine *Software Bill of Materials* (SBOM) listet alle direkten und transitiven Komponenten eines Artefakts auf. Sie ist kein einzelner Sicherheitsmechanismus, sondern der zentrale *Datensatz*, auf dem mehrere Mechanismen aufsetzen: Schwachstellen-Diff, Lizenz-Tracking, Compliance-Nachweise und langfristige Reproduzierbarkeit. Die Implementierung folgt dem in Abbildung 2 skizzierten Lebenszyklus.

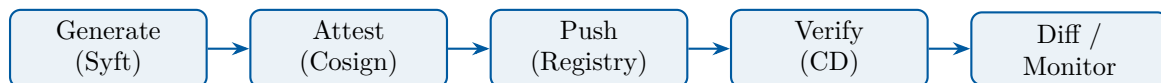


Abbildung 2: SBOM-Lebenszyklus: Erzeugung, Attestation, Verteilung, Verifikation, kontinuierliches Monitoring.

1. Erzeugung mit Syft. Syft (Anchore) inspiziert das gebaute Image schichtweise und erkennt Pakete von Distribution-Manager (`apt`), Sprach-Manager (`pip`, `npm`, `gem`) sowie statisch eingebundene Bibliotheken. Ausgabeformat ist CycloneDX in JSON, weil dies zusätzlich zur reinen Komponentenliste auch *Beziehungen* (`depends-on`, `contains`) und Hashes erfasst – beides ist für spätere Verifikation entscheidend.

2. Attestation mit Cosign. Statt das SBOM lose neben dem Image zu lagern (klassisches `s bom.json` im Release-Asset-Ordner), wird es als signierte *Attestation* an den Image-Digest gebunden: `cosign attest -predicate sbom.cdx.json -type cyclonedx`. Damit ist das SBOM kryptographisch an genau das Image gekettet, das es beschreibt – und manipulierte SBOM-Kopien lassen sich erkennen.

3. Verteilung über die Registry. Cosign legt Attestations als zusätzliche OCI-Manifeste neben dem eigentlichen Image ab; Pull-Clients holen sie auf Wunsch (`cosign download attestation`). Vorteile: keine zweite Distributionspipeline, keine zusätzlichen Zugriffsrechte, keine abweichende Aufbewahrungsdauer.

4. Verifikation vor Deploy. Der CD-Workflow ruft `cosign verify-attestation` mit `-type cyclonedx` und `-certificate-identity-regexp` auf, der nur Attestations der erwarteten CI-Workflow-Identität akzeptiert. Damit wird eine angeheftete SBOM-Manipulation durch einen externen Angreifer mit Registry-Zugriff zur Pipeline-Stufe *vor* dem Rollout sichtbar.

5. Kontinuierliches Monitoring (SBOM-Diff). Der eigentliche operative Wert eines SBOM entsteht erst nach Auslieferung. Bei jedem Build vergleicht ein nachgelagerter Schritt die neue SBOM gegen die letzte gepushte Version desselben Stream-Tags. Drei Fragen werden beantwortet:

1. **Welche Pakete sind hinzugekommen?** Neue transitive Abhängigkeiten sind das primäre Eintrittsfenster für Supply-Chain-Angriffe (vgl. Walkthrough 1 in Abschnitt 4.2).
2. **Welche Versionen haben sich geändert?** Ein Versionssprung über mehrere Major-Releases hinweg ist ein Signal für übersehene Major-Updates und sollte einen Reviewer zwingen, das Changelog zu prüfen.
3. **Sind bekannte CVEs hinzugekommen?** Werkzeuge wie `grype` oder `trivy sbom` korrelieren die SBOM gegen die OSV/NVD-Datenbanken; das ist günstiger als ein kompletter Image-Scan und lässt sich gegen *historische* Artefakte ausführen, die in Produktion noch laufen.

6. Aufbewahrung und Auskunft. SBOMs müssen über die Lebensdauer des Artefakts verfügbar bleiben – für eine Library, die sieben Jahre verkauft wird, also potenziell deutlich länger als ein typischer Container-Lebenszyklus (vgl. die in Abschnitt 2 erwähnte Software-Sustainability-Diskussion). Die Bindung der Attestation an den Image-Digest und die Hinterlegung im Sigstore-Rekor-Log adressiert das auch dann, wenn die ursprüngliche Pipeline-Konfiguration nicht mehr existiert.

Bezug zum Cyber Resilience Act (CRA). Mit dem *Cyber Resilience Act* [7] der EU werden SBOMs ab Ende 2027 für „Produkte mit digitalen Elementen“ eine verpflichtende technische Dokumentation. Hersteller müssen eine SBOM für jedes vermarktete Produkt führen und Behörden auf Anfrage verfügbar machen; bei „important“ bzw. „critical“ Produkten kommen erweiterte Pflichten hinzu (Conformity Assessment, Vulnerability Handling, Coordinated Disclosure). Die hier umgesetzte Pipeline erfüllt die kommenden technischen Anforderungen *by construction*: SBOM wird automatisiert erzeugt, an das Artefakt gebunden, signiert verteilt und ist über das Rekor-Log nachvollziehbar. Die organisatorischen CRA-Pflichten (z. B. 24h-Reporting bei aktiv ausgenutzten Schwachstellen) bleiben außerhalb des Pipeline-Scopes, sind aber durch die SECURITY.md-Policy und das Vulnerability-Disclosure-Verfahren des Repositories vorbereitet.

Grenzen. Eine SBOM ist nur so vollständig wie der Generator. Statisch eingebettete C-Bibliotheken in Wheel-Distributionen, vendoring-basierte Build-Systeme und nachgeladene Plugins entgehen einer rein paketmanager-basierten Erkennung; entsprechende Pakete landen unentdeckt im Image. Die hier verwendete Kombination aus Multi-Stage-Dockerfile mit gepinnten Versionen und Container-Scan (Trivy) auf das fertige Image kompensiert das teilweise, beseitigt aber nicht das grundsätzliche Vollständigkeitsproblem.

5.6 Deploy-Stufe

OIDC-Federation

Cloud-Logins (Azure / AWS) erfolgen via OIDC – keine statischen Cloud-Credentials in Repo-Secrets.

Environments mit Approval

production ist GitHub-Environment mit Pflicht-Reviewer und Wait-Timer.

Cosign-Verify vor Deploy

Vor dem Rollout verifiziert der CD-Workflow Signatur *und* SLSA-Provenance des Image-Digests.

Image-Pinning per Digest

Deployments referenzieren @sha256:..., nicht Tags.

5.7 Auszug aus der CI-Implementierung

Listing 2 zeigt die Job-Topologie und die zentralen Sicherheits-Gates.

```
1 permissions: {}                                # default-deny
2
3 jobs:
4   lint-test:
5     permissions: { contents: read }
6     steps:
7       - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
8         # v4.2.2
9       with: { persist-credentials: false }
10  build-image:
11    needs: [lint-test, sast, sca, secret-scan]
12    permissions:
13      contents: read
14      packages: write
15      id-token: write          # OIDC fuer cosign keyless
16      attestations: write
```

Listing 2: Auszug .github/workflows/ci.yml: Permissions & Pin auf SHA

```
1 cosign verify \
2   --certificate-identity-regexp "https://github.com/$REPO/.github/
3   workflows/.+" \
4   --certificate-oidc-issuer "https://token.actions.githubusercontent.com
5   " \
6   "$IMAGE"
```

```

6 cosign verify-attestation --type slsaprovenance \
7   --certificate-identity-regexp "https://github.com/$REPO/.github/
   workflows/.+" \
8   --certificate-oidc-issuer "https://token.actions.githubusercontent.com
   " \
9   "$IMAGE"

```

Listing 3: Verifikation vor Deploy (cd.yml)

5.8 Mapping Bedrohung → Maßnahme

Bedrohung (Tab. 3)	Maßnahme
Maintainer-Credential-Diebstahl	MFA, signierte Commits, CODEOWNERS-Review
Manipulation Pipeline-Definition	CODEOWNERS auf <code>.github/</code> , Required Checks
Bösartige Dependency	pip-audit, Dependabot, Pinning, SBOM-Diff
Base-Image-Replacement	Pinning + Trivy + Image-Digest in Deploy
Zu weite <code>permissions</code>	default-deny + per-Job opt-in
<code>pull_request_target</code> Leak	Trigger nicht verwendet; PRs ohne Secrets
Tag-Hijack in Registry	Cosign-Verify + Digest-Pinning beim Deploy
Fehlende Provenance	SLSA-Provenance via <code>attest-build-provenance</code>
Long-lived Cloud-Creds	OIDC-Federation, keine statischen Secrets
Fehlendes Approval Production	GitHub Environment Reviewer + Wait-Timer

Tabelle 4: Bedrohungs-Maßnahmen-Matrix.

5.9 Fallstudie: tj-actions/changed-files (März 2025)

Die Action `tj-actions/changed-files` wurde im März 2025 kompromittiert: Angreifer überschrieben mehrere veröffentlichte Tags so, dass sie auf einen manipulierten Commit zeigten, der Secrets aus dem Runner-Memory in die Build-Logs exfiltrierte. Workflows, die die Action ohne SHA-Pinning eingebunden hatten (`tj-actions/changed-files@v44`), zogen automatisch die manipulierte Version. Über zehntausend Repositories waren betroffen, in mehreren Fällen wurden GitHub-PATs und Cloud-Credentials öffentlich auf GitHub-Logs sichtbar.

Der Vorfall illustriert exakt die Bedrohungsclass *Source/Build-T* aus Tabelle 3. Die in dieser Arbeit umgesetzte Maßnahme – durchgängiges Pinning aller Actions auf den Commit-SHA mit einem nachgestellten Versions-Kommentar – hätte den Angriff vollständig neutralisiert: Selbst wenn der Tag `v44` auf einen böartigen Commit umgehängt wird, verweist der Pin `tj-actions/changed-files@<sha> # v44` weiterhin auf die auditierte Version. Sigstore/Cosign hätte die Auswirkung zusätzlich begrenzt, da das resultierende Artefakt keine gültige Signatur unter der OIDC-Identität des Workflows getragen hätte.

6 Evaluation

6.1 SLSA-Einstufung

Supply-chain Levels for Software Artifacts (SLSA) definiert in Version 1.0 [1] eine ordinale Skala für die Vertrauenswürdigkeit der Build-Plattform und des Quellcode-Prozesses. Die Skala besteht aus zwei orthogonalen Tracks (*Build* und *Source*); die jeweiligen Levels sind kumulativ, d. h. jedes höhere Level fordert die Anforderungen aller niedrigeren mit. Im Folgenden werden die Levels zusammengefasst, mit der konkreten Umsetzung im untersuchten System abgeglichen und die Begründung für *nicht* erreichte höhere Stufen transparent gemacht.

Build-Track.

L0

Keine besonderen Anforderungen. Der Build kann lokal, manuell, ohne jede Provenance erfolgen.

L1

Provenance existiert in irgendeiner Form – der Build ist also reproduzierbar dokumentiert. Inhalt und Authentizität der Provenance werden noch nicht abgesichert.

L2

Provenance ist *signiert* und wird von einem *gehosteten Build-Service* erzeugt. Der Build läuft also nicht mehr auf einer Entwickler-Workstation; der Build-Service kann Provenance an seine eigene Identität binden.

L3

Der Build-Service garantiert zusätzlich *Non-Forgeable Provenance* und *Isolation* zwischen Builds: Provenance kann nicht durch den Build-Schritt selbst manipuliert werden, und ein Build kann nicht mit Artefakten oder Geheimnissen vorheriger Builds interagieren.

Die hier umgesetzte Pipeline erreicht **Build L3**. Konkret:

- GitHub-hosted Runner (ubuntu-24.04) ist ein gehosteter Service; jeder Job läuft in einer frisch provisionierten VM, was die Isolationsanforderung erfüllt (*ephemeral builder*).
- Provenance wird mit `actions/attest-build-provenance` erzeugt und über die OIDC-Identität des Workflows signiert; das Signaturmaterial liegt nie im für die Build-Schritte zugreifbaren Speicher (Non-Forgeable).
- Das Build-Skript (`Dockerfile` und Workflow) ist versioniert, mit Branch-Protection und CODEOWNERS-Review geschützt.

Source-Track.

L1

Quellcode ist versioniert (z. B. Git); Änderungen sind rückverfolgbar.

L2

Zusätzlich *verifizierte Geschichte*: Commits sind kryptographisch signiert; der Repository-Provider verifiziert Identitäten beim Push.

L3

Zusätzlich *Two-person review* und Schutz vor Force-Push / History-Rewrite.

Die Pipeline erreicht **Source L3**: `main` ist branch-protected mit CODEOWNERS-Review (Vier-Augen), Force-Push und History-Rewrite sind serverseitig deaktiviert, signierte Commits sind Pflicht-Status-Check.

Warum nicht höher? SLSA v1.0 definiert offiziell nur Build-L0 bis L3 und Source-L1 bis L3; es gibt also kein „L4“, das hier zu erreichen wäre. Konzeptionell relevant für eine Studienarbeit ist dennoch die Diskussion, welche *darüber hinausgehenden* Eigenschaften ein „strenger als L3“-Setup hätte:

1. **Hermetische Builds** – der Build hat keinen Netzwerkzugriff zur Build-Zeit; alle Inputs sind vorab in einem Lockfile mit Hash-Pinning erfasst. Die hier verwendeten Standard-pip-Installs verletzen diese Eigenschaft, da der Resolver Indizes anfragt. Eine vollständige Härtung würde `pip install --require-hashes` mit erzeugten `hashin`-Lockfiles und einem Egress-Restricted Runner erfordern.
2. **Reproduzierbare Builds** – bit-für-bit identische Artefakte aus identischen Inputs. Container-Images sind hier notorisch problematisch (Build-Zeitstempel, Layer-IDs, Mtime in Tar-Headern). Werkzeuge wie `kaniko` mit `--reproducible` oder `buildkit`'s `SOURCE_DATE_EPOCH` adressieren das, sind aber im Standard-docker/build-push-action-Setup nicht aktiviert.
3. **Multi-Party-Builds** – zwei unabhängige Build-Plattformen produzieren dasselbe Artefakt; Abweichungen sind ein Alarmsignal. Das ist außerhalb der Reichweite einer Pipeline auf einem einzigen Provider.

Diese Eigenschaften werden im Ausblick (Kapitel 7) als weiterführende Forschungsrichtungen benannt. Für den Studienarbeit-Scope ist die kombinierte Erreichung von Build-L3 + Source-L3 ein bewusst gesetzter Höchststand, der mit handelsüblichen Mitteln (GitHub Actions + Sigstore) ohne kommerzielle Add-ons darstellbar ist.

Verifikation der Einstufung. Die Selbst-Einstufung wird durch externe Werkzeuge geengeprüft: OpenSSF Scorecard (Abschnitt 6.2) bewertet einzelne SLSA-relevante Checks objektiv; `actions/attest-build-provenance` produziert maschinenlesbare Provenance, die mit `slsa-verifier` gegen die deklarierten Anforderungen geprüft werden kann.

6.2 OpenSSF Scorecard

Der Scorecard-Workflow läuft wöchentlich. Erwartete Scores $\geq 8/10$ für *Branch-Protection*, *Pinned-Dependencies*, *Token-Permissions*, *Signed-Releases*, *SAST*, *Vulnerabilities*.

6.3 Coverage der STRIDE-Kategorien

Tabelle 5 fasst zusammen, welche STRIDE-Kategorien durch die Maßnahmen mitigiert sind.

Kategorie	Mitigiert	Wesentliche Maßnahme
Spoofing	vollständig	OIDC, signierte Commits, Cosign
Tampering	weitgehend	Pinning, Provenance, Verify-Gate
Repudiation	vollständig	Provenance + Rekor-Transparency-Log
Information Disclosure	weitgehend	gitleaks, least-privilege Tokens
Denial of Service	teilweise	concurrency-cancel, Cache-Strategie
Elevation of Privilege	vollständig	default-deny permissions, Environments

Tabelle 5: STRIDE-Coverage durch implementierte Maßnahmen.

6.4 Validierung gegen OWASP Top 10 CI/CD

Zur Plausibilisierung der STRIDE-Analyse wird die abgeleitete Maßnahmenliste zusätzlich gegen die OWASP-Top-10-CI/CD-Risiken [6] geprüft. Tabelle 6 zeigt die Abdeckung.

Nr.	OWASP-Risiko	Wesentliche Maßnahme
1	CICD- Insufficient Flow Control Mechanisms	Branch-Protection, CODEOWNERS, Environments mit Approval
2	CICD- Inadequate Identity & Access Mgmt	OIDC-Federation, least-privilege permissions
3	CICD- Dependency Chain Abuse	Pinning, pip-audit, Dependabot, SBOM-Diff
4	CICD- Poisoned Pipeline Execution (PPE)	Kein <code>pull_request_target</code> , CODEOWNERS auf <code>.github/</code>
5	CICD- Insufficient PBAC	Per-Job permissions, Environment-Reviewer
6	CICD- Insufficient Credential Hygiene	gitleaks, Push-Protection, Cosign keyless (keine Long-lived Keys)
7	CICD- Insecure System Configuration	Scorecard wöchentlich, Action-Pinning, default-deny Permissions
8	CICD- Ungoverned Usage of 3rd-Party Services	Kuratierte Action-Liste, SHA-Pin, Renovate/Dependabot Audit
9	CICD- Improper Artifact Integrity Validation	Cosign-Verify + SLSA-Provenance vor Deploy, Digest-Pinning
10	CICD- Insufficient Logging & Visibility	SARIF-Upload, Rekor Transparency-Log, Audit-Logs (org-level)

Tabelle 6: Abdeckung der OWASP-Top-10-CI/CD-Risiken durch die implementierte Pipeline.

Alle zehn Risikoklassen werden adressiert. CICD-10 ist organisatorisch abhängig (Audit-Logs auf Org-Ebene erfordern entsprechende GitHub-Enterprise-Konfiguration) und damit nur teilweise innerhalb der Pipeline selbst mitigierbar.

6.5 Restrisiken

- **Plattform-Kompromittierung** (GitHub, Sigstore): nicht durch die Pipeline mitigierbar; Vertrauensanker.
- **Insider mit Admin-Rechten** kann Branch-Protection deaktivieren. Mitigation: Audit-Logging, Org-Policies.
- **Zero-Day in transitiver Abhängigkeit** – pip-audit erkennt nur bekannte CVEs.
- **Secret-Exfiltration via Build-Logs** bei fahrlässigem `echo`; nur durch Code-Review und `add-mask` mitigierbar.

6.6 Zielkonflikt Geschwindigkeit vs. Assurance

Strikte Gates (HIGH/CRITICAL fail, Coverage 80 %, manuelle Approvals) verlangsamen Deployments. Die Pipeline mildert dies durch parallele Jobs, GHA-Caches und **concurrency: cancel-in-progress** für PRs. Production behält bewusst eine Approval-Hürde – Geschwindigkeit darf hier nicht über Assurance dominieren.

6.7 Faktor Mensch und Anreize

Technische Mechanismen entfalten ihre Wirkung nur, wenn die beteiligten Akteure – Entwickler, Reviewer, Plattform-Administratoren – sie auch konsequent anwenden. Anderson formuliert im Security Engineering Framework die These, dass Sicherheit nicht primär an fehlenden Mechanismen scheitert, sondern an fehlerhaft gesetzten *Anreizen*: Die Person, die schützen *soll*, hat oft nicht denselben Nutzen vom erfolgreichen Schutz wie die Person, die im Schadensfall den Verlust trägt. Dieser Anreizbruch ist im DevSecOps-Kontext besonders prägnant, weil das Liefer- und das Sicherheits-Ziel dieselbe Pipeline teilen. Die folgenden Spannungsfelder sind in der Praxis besonders relevant.

Geschwindigkeit vs. Reibung. Sicherheits-Gates wie Coverage-Schwellen, signierte Commits oder Manual-Approval-Schritte verlangsamen den Feedback-Zyklus. Wird die Reibung als unverhältnismäßig empfunden, neigen Teams dazu, Schutzmaßnahmen zu umgehen – etwa durch `[skip ci]`-Commits, `-no-verify` bei pre-commit-Hooks oder das pauschale Akzeptieren von HIGH-Findings als „false positive“. Die Pipeline mildert dies, indem sie schnelle Jobs (Lint, Tests) parallel zu langsamen (CodeQL, Trivy) ausführt und PRs bei neuen Commits via **cancel-in-progress** abbricht. Die wichtigere Beobachtung ist jedoch struktureller Natur: Wenn der lokale **pre-commit**-Hook dieselben Checks ausführt wie die CI, fängt ein Entwickler Verstöße bereits vor dem Push – der Schutz wird damit zur *schnellsten* Option, nicht zur langsamsten.

Ownership-Diffusion. „Security ist Aufgabe des Security-Teams“ ist ein verbreitetes Anti-Pattern; in Conway’s Sinn spiegelt jede Pipeline die Verantwortungsstruktur der Organisation wider, die sie betreibt. Eine zentrale Security-Funktion mit Veto-Recht erzeugt typischerweise eine Zwei-Klassen-Pipeline: schnelle „Dev“-Builds und langsame

„Security“-Audits, die sich entkoppelt entwickeln. CODEOWNERS verschiebt das Eigentum für sicherheitskritische Pfade (`.github/`, `Dockerfile`) explizit auf die Pipeline-Verantwortlichen und macht Reviews damit zum unumgehbaren Default; gleichzeitig wird das Reviewer-Recht nicht zentralisiert, sondern an die fachlich nächsten Personen delegiert. Die OpenSSF-Scorecard liefert eine *externe* Bewertung, die Diskussionen über Posture-Defizite versachlicht und sie aus der politischen in die operative Sphäre verschiebt.

Alarmmüdigkeit. Eine Pipeline, die bei jedem Push fünfzig Findings produziert, wird ignoriert – ein Muster, das in der Forschung als *alert fatigue* seit Jahren empirisch belegt ist. Die Implementierung adressiert dies durch die Trennung in Build-Breaker (`HIGH, CRITICAL`, mit `ignore-unfixed: true`) und reine Reportings via SARIF. Damit bleibt jeder Build-Abbruch handlungsrelevant, und Findings ohne unmittelbaren Fix verschwinden nicht aus der Sicht (Code-Scanning-View), erzwingen aber keinen Stillstand. Wichtig ist zudem die *Reproduzierbarkeit lokal*: Wenn ein Entwickler denselben Bandit- oder Trivy-Lauf auf der Workstation reproduzieren kann, sinkt die Hemmschwelle, ein Finding tatsächlich zu adressieren, massiv – daher das `pre-commit`-Setup mit denselben Tools.

Anreiz für Maintainer. Dependabot erzeugt Updates als kleine, isolierte PRs, die die volle Pipeline durchlaufen. Damit wird das Aktualisieren von Abhängigkeiten zur *billigsten* statt zur teuersten Option – ein klassischer Anreiz-Hebel im Sinne des Security-Engineering-Frameworks (Tab. 1). Der Hebelpunkt ist subtil: Ohne Automatisierung ist ein Dependency-Update ein bewusst zu treffender Aufwand, der mit dem Risiko eines Regressionsfehlers gegen den Nutzen einer hypothetischen zukünftigen CVE-Vermeidung abgewogen werden muss. Mit Automatisierung kehrt sich das Kosten/Nutzen-Verhältnis um: Den PR *nicht* zu mergen erfordert eine aktive Begründung.

Asymmetrie zwischen Angreifer und Verteidiger. Eine in der Lehrveranstaltung mehrfach betonte Beobachtung ist die *strukturelle Asymmetrie*: Der Angreifer muss eine Lücke finden, der Verteidiger alle schließen. In CI/CD potenziert sich dieser Effekt, weil die Pipeline Vertrauensanker für eine ganze Reihe nachgelagerter Systeme ist – ein einziger Punkt, an dem ein Angreifer transitiv die Produktion kompromittiert. Die Antwort der Pipeline darauf ist nicht Perfektion, sondern Verteidigung in Tiefe: SHA-Pinning (verhindert Tag-Retag), least-privilege Permissions (begrenzt Sprengkraft eines erfolgreichen PPE), Cosign-Verify (erkennt Tag-Hijack post-build), Manual-Approval (letzte menschliche Hürde). Jede Einzelmaßnahme ist umgehbar; die Kombination ist es deutlich weniger.

Verantwortungs-Verschiebung durch „Shift Left“. Der DevSecOps-Slogan „shift left“ verschiebt Sicherheitsverantwortung von einem nachgelagerten Audit-Team zu den Entwicklern selbst. Das ist intendierter Effekt – Bugs werden früher und billiger behoben – erzeugt aber eine Kehrseite: Entwickler werden für Entscheidungen verantwortlich, für die sie weder ausgebildet sind noch die Zeit zugewiesen bekommen. Ein Bandit-Finding zu einer `subprocess.shell=True`-Verwendung erfordert eine Sicherheitsbewertung im Kontext der Aufrufkette. Werden solche Findings ohne unterstützendes Material allein dem Entwickler überlassen, ist die Standardreaktion das Stummschalten per `# nosec`-Kommentar. Die Pipeline kompensiert das nur teilweise (Bandit-Findings landen als SARIF im Code-Scanning-View und sind damit auch für andere Reviewer sichtbar) – die

verbleibende Schulungs- und Begleitarbeit ist eine organisatorische Investition, die kein Tool ersetzt.

Falsche Sicherheit durch Zertifikate. Eine grüne Pipeline und ein hoher Scorecard-Wert können den Eindruck erwecken, ein Repository sei „sicher“. Schramm verweist in der Vorlesung explizit auf das Phänomen *Security Theatre* (nach Schneier): sichtbare, aber wirkungsschwache Maßnahmen werden wirksamen, aber unsichtbaren vorgezogen, weil sie das Vertrauen der Stakeholder herstellen. Die OpenSSF-Scorecard ist anfällig für genau dieses Muster, da sie objektiv messbare Indikatoren bewertet (Existenz einer SECURITY.md, Action-Pinning, Branch-Protection-Konfiguration), nicht aber die *Qualität* der gelebten Praxis: Ob die SECURITY.md tatsächlich gelesen wurde, ob CODEOWNERS-Reviews substanziell oder rubber-stamp sind, ob die Build-Breaker-Findings wirklich behoben oder nur weggesilenced werden – all das fällt durch das Raster. Die Pipeline ist daher nicht als Endpunkt, sondern als *Mindeststandard* zu lesen, auf dem eine kontinuierliche Sicherheitskultur aufsetzen muss.

Insider als Grenzfall. Alle hier diskutierten Mechanismen unterstellen, dass die Pipeline-Konfiguration selbst durch reguläre Reviews geschützt ist. Ein Administrator mit Force-Push-Recht oder das Recht, Branch-Protection zu deaktivieren, kann das gesamte Schutzgerüst aushebeln, ohne dass eine einzige technische Komponente versagt. Das ist kein Defekt der Pipeline, sondern eine fundamentale Eigenschaft jedes Systems mit unteilbarer Admin-Identität. Die Mitigation liegt außerhalb der Pipeline: rollengetrennte Org-Strukturen, Audit-Logging mit unabhängigem Empfänger, regelmäßige Posture-Reviews durch externe Stellen. Diese Maßnahmen werden in den Restrisiken (Abschnitt 6.5) als bewusst nicht durch die Pipeline adressierbar geführt.

Zusammenfassung. Die diskutierten Spannungsfelder zeigen, dass die untersuchte Pipeline *nicht* nur ein Werkzeug-Stapel ist, sondern eine Antwort auf eine Anreizlage: Sie macht Sicherheit zur billigsten verfügbaren Option, sie verteilt Verantwortung über die Organisation, sie entkoppelt Build-Breaker von Reportings, und sie hält bewusst eine menschliche Hürde an der kritischsten Stelle (Production-Approval) aufrecht. Diese Entscheidungen sind in den Quadranten *Incentives* des SE-Frameworks (Tab. 1) verortet und damit gleichberechtigt zu den technischen Mechanismen.

7 Fazit und Ausblick

Die Arbeit zeigt, dass sich eine moderne CI/CD-Pipeline mit zugänglichen Mitteln (GitHub Actions, Sigstore, Trivy, Syft) systematisch entlang einer STRIDE-Analyse absichern lässt und dabei nachweisbare Eigenschaften gemäß SLISA erreicht. Wesentlich ist nicht das einzelne Tool, sondern die Kombination: *least-privilege Identitäten, gepinnte Abhängigkeiten, signierte Artefakte mit Provenance, verifizierte Deployments*.

Weiterführende Arbeiten könnten:

- die Übertragbarkeit der Maßnahmen auf GitLab CI quantitativ vergleichen,

- Policy-as-Code (z. B. Kyverno, Sigstore Policy Controller im Cluster) als zusätzliche Verifikationsschicht ergänzen,
- formale Analysen zu Pipeline-Konfigurationen (z. B. `zizmor`, `actionlint`) mit Beweisbarkeit der *permissions*-Minimalität verbinden.

Literatur

- [1] OpenSSF, *Supply-chain Levels for Software Artifacts (SLSA) v1.0*, <https://slsa.dev/spec/v1.0/>, abgerufen 2026.
- [2] NIST, *Secure Software Development Framework (SSDF) Version 1.1*, SP 800-218, 2022.
- [3] Microsoft, *The STRIDE Threat Model*, Microsoft Learn, abgerufen 2026.
- [4] OpenSSF, *Scorecard*, <https://github.com/ossf/scorecard>, abgerufen 2026.
- [5] Sigstore Project, *Cosign / Fulcio / Rekor*, <https://www.sigstore.dev/>, abgerufen 2026.
- [6] OWASP, *Top 10 CI/CD Security Risks*, <https://owasp.org/www-project-top-10-ci-cd-security-risks/>, abgerufen 2026.
- [7] Europäische Union, *Cyber Resilience Act* (Verordnung (EU) 2024/2847), <https://eur-lex.europa.eu/eli/reg/2024/2847/oj>, 2024.